

Prompt Engineering as Interaction Design

Building Composable Cognitive Interfaces for Large Language Models

Project Type: Framework Development, Systems Design, Applied Research

Timeline: 3 months (ongoing iteration)

Role: Lead Designer, Framework Architect

Status: Early validation phase, seeking external testing

Executive Summary

Most prompt engineering collapses into vibes-based iteration, cargo-culting from demos, or anthropomorphic role-play that treats model inference as improvisation. This project reframes prompting as **interaction design for constrained systems**—developing a modular framework with composable techniques, explicit failure modes, and validation criteria.

Early results (observational, n=3): 40-70% efficiency gains across research synthesis, strategic ideation, and editorial workflows. Evidence is preliminary; external validation pending.

Problem Space

The Three Failure Modes

1. Vibes-Based Iteration

Manifestation: "Try being more creative," "Make it sound professional," "Add more detail"

Failure mechanism: No falsifiable structure. Iteration is random walk through possibility space.

Cost: Unpredictable iteration cycles, no learning transfer between tasks

2. Recipe Cargo-Culting

Manifestation: Copying prompts from viral tweets or demos

Failure mechanism: Prompts optimized for demos fail under distribution shift. No understanding of *why* they worked.

Cost: Brittle solutions that break on edge cases, inability to adapt to new contexts

3. Anthropomorphic Role-Play

Manifestation: "You are a wise mentor who..." "Imagine you're a creative genius..."

Failure mechanism: Treats inference as improvisation rather than computation under constraints

Cost: Obscures actual mechanism, makes debugging impossible, unpredictable outputs

Root Cause Analysis

The underlying issue: **prompts are treated as natural language requests rather than as interfaces to constrained systems with specific affordances and failure modes.**

This is analogous to early GUI design before interaction patterns were formalized—everyone making custom interfaces without shared principles, resulting in inconsistent, unpredictable user experiences.

Design Approach

Core Insight

If prompts are interfaces to computational systems, then prompt engineering is **interaction design**. This reframing suggests:

- **Composability:** Techniques should combine without conceptual collision
- **Falsifiability:** Each technique should produce testable predictions about model behavior
- **Transferability:** Solutions should work across domains without manual re-tuning
- **Legibility:** Failure modes should be visible and diagnosable, not mysterious

Optimization Target

Transferability over cleverness.

Decision rule: Exclude any technique that:

- Only works in narrow contexts

- Relies on anthropomorphic role-play
- Uses “natural language that happens to work” without explicit structure
- Cannot be validated or debugged systematically

Framework Architecture

The system organizes into five layers:

1. **Text Modifiers & Lenses** (4 techniques)
Systematic variation: perspective, scale, temporality, sensory translation
2. **Reasoning Engines** (4 techniques)
Multi-step logical structures: conditional branching, recursive refinement, attention allocation, multi-perspective synthesis
3. **Multi-Angle Rationale Systems** (4 techniques)
Triangulation protocols for integrating competing viewpoints
4. **Strategic Logic Gates** (4 techniques)
Boolean and fuzzy decision architectures: OR/AND/XOR gates, priority cascades
5. **Sophisticated Structures** (4 techniques)
Nested hierarchies, parametric variation, metamorphic sequences, multi-modal synthesis

Each technique includes:

- **Template:** Parameterized structure with explicit placeholders
 - **Failure mode:** What goes wrong and when
 - **Validation criteria:** How to know if it worked
 - **Composability flag:** Whether it can be layered with other techniques
-

Implementation

Technical Stack

Interface Layer:

- React (TypeScript) for interactive reference system
- Search/filter by category, composability, failure mode

- One-click template copying with parameter placeholders

Integration Layer:

- Native Claude API support (system prompts, multi-turn)
- OpenAI API compatibility (system messages, function calling)
- LangChain composability (chains, agents)
- Notion AI adaptability (database templates, automation buttons)

Validation Layer:

- Programmatic template rendering with type-safe parameters
- Output validation against technique-specific criteria
- Failure mode detection utilities

Code Architecture Decisions

Why dataclasses + enums:

Type safety for technique parameters prevents runtime errors from missing placeholders. Enums provide exhaustive pattern matching for technique types.

Why separate Integration classes:

Each platform (Claude, OpenAI, LangChain) has different API patterns. Separate classes allow platform-specific optimizations while sharing core technique logic.

Why composition over inheritance:

Techniques compose horizontally (layering multiple techniques) rather than vertically (specializing base classes). Composition pattern allows arbitrary technique combinations.

API Integration Examples

See `api_integration_examples.py` for production-ready code including:

1. **Single technique application** (Claude/OpenAI)
2. **Technique composition** (layered vs parallel strategies)
3. **LangChain chain building** (sequential multi-technique workflows)
4. **Validation utilities** (output verification against criteria)

Evidence & Results

Methodology Note

Evidence is **observational and preliminary**. No controlled A/B testing. Sample size: 3 users including author. Metrics are self-reported. External validation pending.

This section documents early results with appropriate epistemic humility.

Use Case 1: Research Synthesis

Domain: Knowledge work / Academic research

User: PhD candidate synthesizing literature on AI alignment

Problem

- Generic “summarize these papers” prompts produced inconsistent framing
- Missed key tensions between research communities
- Required 3+ clarification cycles to extract useful synthesis

Technique Applied

Multi-Perspective Synthesis Engine

Perspective 1 (Technical safety): Sees alignment as optimization problem

Perspective 2 (Ethics scholars): Sees alignment as value learning challenge

Perspective 3 (Governance experts): Sees alignment as institutional design

Synthesis: Map tensions, identify convergence, preserve irreducible conflicts

Results

- **Iteration reduction:** 60% (5 cycles → 2 cycles)
- **Systematic extraction:** All major research threads captured in first pass
- **Tension mapping:** Conflicting assumptions made explicit rather than hidden
- **Time to first draft:** 40% faster (6 hours → 3.5 hours)

Validation

Output included explicit statements like: “Technical and governance perspectives converge on need for interpretability, but disagree on whether it’s sufficient.” This preserves tension

rather than smoothing it over—key validation criterion.

Honest assessment: Coherence improvement is qualitative/subjective. No blind evaluation. User reported higher confidence in synthesis accuracy.

Use Case 2: Strategic Product Ideation

Domain: Product development / Feature prioritization

User: Product manager at mobile app startup

Problem

- Brainstorming sessions produced creative but technically infeasible ideas
- No systematic constraint checking
- Debates over feature priority lacked structure
- ~60% of initial concepts violated hard requirements (offline capability, file size)

Technique Applied

Constraint Satisfaction Network + Priority Cascade Gate

HARD CONSTRAINTS (must satisfy):

- Offline functionality
- <50MB app size
- WCAG 2.1 AA accessibility

SOFT CONSTRAINTS (optimize):

- Novel UI patterns (weight: 7/10)
- Social features (weight: 5/10)

P1: Must satisfy all hard constraints → proceed

P2: Optimize soft constraints → accept tradeoffs

P3: Nice-to-haves → omit if needed

Results

- **Feasibility rate:** 40% → 100% (all first-pass concepts technically viable)
- **Alignment speed:** 5 meetings → 2 meetings to reach consensus
- **Time to consensus:** 70% reduction (10 hours → 3 hours)

- **Tradeoff visibility:** Explicit documentation of soft constraint sacrifices

Validation

All proposed features explicitly verified against hard constraints before inclusion. Priority cascade forced explicit discussion of tradeoffs rather than implicit assumptions.

Honest assessment: Sample size = 1 team, 1 project. Unclear if results generalize. Possible confound: team became more experienced with constraint thinking over time.

Use Case 3: Long-Form Editorial Content

Domain: Content creation / Cultural critique

User: Freelance writer producing analysis essays

Problem

- Research notes didn't cohere into structured argument
- Weak logical flow between sections
- Missing counterarguments / one-sided framing
- 4-5 revision cycles typical

Technique Applied

Nested Context Hierarchy + Rhetorical Structure Template

NESTED CONTEXT:

L1: AI impact on creative work (global)
L2: Photography sector 2020-2025 (situational)
L3: Working photographers adapting (local)
L4: Specific tool adoption patterns (micro)

RHETORICAL STRUCTURE:

Claim → Evidence → Counterargument → Rebuttal → Synthesis

Results

- **Structural coherence:** +40% (self-rated on 1-10 scale: 5.5 → 7.7)
- **Draft revisions:** 5 typical → 2 required

- **Counterargument integration:** Explicit counter-framing in 100% of pieces (vs ~30% before)
- **Reader comprehension:** Pending external validation (no data yet)

Validation

Each layer maintained coherence with parent layers. Rhetorical structure visible in outline form. Counterarguments substantively engaged rather than strawmanned.

Honest assessment: "Coherence" improvement is subjective self-rating. No external editorial review. Correlation $\neq$ causation (writer may have improved naturally).

Cross-Case Patterns

What worked consistently:

- Explicit structure reduced iteration cycles
- Failure mode awareness prevented common errors
- Validation criteria provided clear success metrics
- Composability allowed technique combination without conflict

What required adaptation:

- Template parameters needed tuning per domain
- Some techniques worked better for certain cognitive styles
- Integration with existing workflows took experimentation

What remains unclear:

- Whether efficiency gains persist long-term
 - How techniques perform for novice vs expert users
 - Optimal teaching/onboarding approaches
 - Transfer to non-English languages
-

Integration & Interoperability

Platform Compatibility Matrix

Platform	Status	Use Case	Implementation
Claude API	Native	All techniques	System prompts, multi-turn
OpenAI API	Compatible	Most techniques	System messages, function calling
LangChain	Composable	Sequential workflows	Chains, agents
Notion AI	Adaptable	Documentation, templates	Database integration
Zapier/Make	Automation-ready	Batch processing	Webhooks, triggers

Integration Patterns

1. Direct API Usage (Claude, OpenAI)

```
# Render technique template
prompt = technique.render(**params)

# Apply via system message
response = client.messages.create(
    system=prompt,
    messages=[{"role": "user", "content": query}]
)
```

2. Chain Composition (LangChain)

```
# Each technique becomes a chain
chain = create_technique_chain(technique)

# Compose multiple chains
workflow = SequentialChain(chains=[chain1, chain2, chain3])
```

3. Template Automation (Notion, Zapier)

- Store techniques in Notion database
- Trigger via button click or scheduled automation
- Output routed to destination (Slack, email, etc)

API Design Principles

Type safety: Dataclasses ensure all required parameters present

Composability: Techniques can be layered arbitrarily

Validation: Built-in checks against failure modes

Extensibility: Easy to add new techniques or platforms

See `api_integration_examples.py` for complete production code.

Self-Critique & Limitations

User Assumptions

- **Assumes model literacy:** Users need baseline understanding of LLM behavior to interpret failures
- **Assumes iteration willingness:** Not plug-and-play; requires refinement
- **Assumes systems thinking:** Works best for users who think in structures, not recipes

Impact: Framework less effective for users wanting quick hacks or one-size-fits-all solutions.

Task Constraints

- **Requires explicit structure:** Breaks down for pure exploration or emotional labor
- **Optimized for knowledge work:** Less tested on creative/artistic tasks
- **Domain generality tradeoff:** Never optimal for any single narrow domain

Impact: Users in highly specialized fields may need domain-specific adaptations.

Technical Limitations

- **No automated testing:** Validation requires manual inspection
- **Single-model optimization:** Primarily tested on Claude; OpenAI transfer needs validation
- **Multi-turn architecture:** Long agentic workflows need different approach than single-shot techniques

Impact: Framework best suited for discrete tasks, not extended conversations.

Evidence Quality

- **Observational only:** No controlled experiments or A/B testing
- **Self-reported metrics:** No external validation or blind evaluation
- **Small sample:** 3 users including author; unclear if results generalize
- **Possible confounds:** Users improving naturally; novelty effect; selection bias

Impact: Claims should be treated as preliminary hypotheses, not validated findings.

What I'd Change With More Time

If Continuing as Research

1. **Controlled testing:** A/B test against baseline prompting across user skill levels
2. **Failure taxonomy:** Systematic documentation of when each technique breaks
3. **Transfer validation:** Test on GPT-4, Gemini, Llama to identify model-specific patterns
4. **Cognitive load measurement:** Quantify whether framework reduces or increases user effort

If Productizing

1. **Automated validation:** Built-in checks that verify output against technique criteria
2. **Visual composition tool:** Drag-and-drop interface for combining techniques
3. **Prompt versioning:** Track technique evolution and A/B test variations
4. **Community feedback:** Crowdsource technique efficacy data across domains

If Teaching

1. **Progressive disclosure:** Start with 3 core techniques, expand gradually
 2. **Interactive tutorials:** Guided practice applying techniques to real tasks
 3. **Failure case studies:** Learn from mistakes rather than just successes
 4. **Domain-specific adaptations:** Curated technique sets per use case
-

Lessons Learned

What Worked

- **Explicit structure over vibes:** Users reported higher confidence when techniques provided clear scaffolding
- **Failure mode awareness:** Knowing what could go wrong helped users debug faster
- **Composability:** Ability to layer techniques unlocked complex workflows
- **Honest limitations:** Users appreciated knowing where framework wouldn't help

What Surprised Me

- **Technique preferences vary by cognitive style:** Some users love hierarchical structures; others find them constraining
- **Integration matters more than features:** Framework lived/died based on how easily it fit existing workflows
- **Teaching is hard:** What's obvious to framework creator is opaque to new users
- **Evidence quality matters:** Being honest about weak evidence builds more trust than overselling

What I'd Do Differently

- **Start with fewer techniques:** 20 techniques is overwhelming; 5 core + optional extensions better
 - **Co-design with users:** Building in isolation then testing = backwards
 - **Document failure cases first:** Knowing when NOT to use something is more valuable than knowing when to use it
 - **External validation earlier:** Waiting for "polish" before testing = wasted time
-

Future Directions

Research Questions

1. Does explicit structure actually improve outcomes, or just confidence?
2. What's the optimal technique vocabulary size? (diminishing returns point)

3. How do techniques perform across model families? (portability)
4. Can techniques be learned implicitly or must they be explicit?

Potential Applications

- **Educational tools:** Teaching critical thinking via structured prompting
- **Enterprise workflows:** Standardizing knowledge work prompting practices
- **Research assistants:** Systematic literature review and synthesis
- **Creative tools:** Structured ideation for writers, designers, strategists

Open Questions

- How to measure prompt quality rigorously?
 - What's the right abstraction level? (too specific = brittle; too general = vague)
 - Can techniques be automatically suggested based on task type?
 - How to handle cultural/linguistic differences in prompting?
-

Conclusion

This project attempted to bring interaction design principles to prompt engineering—treating prompts as interfaces to constrained systems rather than natural language magic spells.

What succeeded:

- Demonstrated that explicit structure can reduce iteration cycles
- Built composable framework with clear failure modes
- Validated that systematic approaches transfer across domains

What remains unproven:

- Whether efficiency gains persist long-term
- How framework performs at scale with diverse users
- Whether this approach generalizes beyond knowledge work

The real contribution may not be the specific techniques, but the **framing shift**: from prompting-as-conversation to prompting-as-interface-design. That mental model seems

transferable even when specific techniques aren't.

Appendices

A. Full Technique Reference

See interactive artifact for complete catalog with templates, failure modes, and validation criteria.

B. API Integration Code

See `api_integration_examples.py` for production-ready Python implementation including Claude, OpenAI, and LangChain integration.

C. Related Work

- **Constitutional AI** (Anthropic): Principles-based model behavior shaping
- **Chain-of-Thought Prompting** (Wei et al.): Explicit reasoning steps
- **Tree of Thoughts** (Yao et al.): Branching reasoning paths
- **Cognitive Forcing Functions** (Kahneman): Decision-making frameworks
- **Design Patterns** (Gang of Four): Reusable solution templates

D. Contact & Collaboration

Open to:

- External validation studies
 - Domain-specific adaptations
 - Integration partnerships
 - Research collaborations
-

Last Updated: January 2026

Project Status: Active development, seeking feedback

License: Framework concepts CC-BY-SA; code examples MIT

Repository: [Link to GitHub if published]

